

Solving Combinatorial Optimization Problems using Simulated Annealing

PROJECT REPORT

IE211 - Programming & Computing Lab

Team Members:

Harshit Singh Bhandari - 24B4506

Shivam Soni - 24B4525

Adit Prabhu - 24B4535

**Department of Industrial Engineering & Operations Research
IIT Bombay**

Contents

1	Introduction	2
2	Methodology	2
2.1	What is Simulated Annealing?	2
2.2	Advantages of Simulated Annealing	3
2.3	Implementation Architecture	4
2.3.1	The Abstract State Class	4
2.3.2	The Annealer Class	5
3	Application I: Traveling Salesperson Problem (TSP)	8
3.1	Problem Context	8
3.2	Limitations of Classical Optimization Methods	8
3.3	Implementation Details	8
3.3.1	Path Representation	8
3.3.2	Neighborhood Structure	8
3.3.3	Objective Function	9
3.3.4	Greedy Search	10
3.4	Results and Performance	11
3.5	Limitations and Tradeoffs	11
3.6	Conclusion	11
4	Application II: Portfolio Optimization	12
4.1	Problem Context	12
4.2	Limitations of Classical Optimization Methods	12
4.3	Why Simulated Annealing is used	12
4.4	Portfolio Representation	13
4.5	Constraint Handling	13
4.6	Neighborhood Structure	13
4.7	Objective Function	13
4.8	Results	14
4.9	Impact of the Quench Phase	15
5	Application III: Community Detection	16
5.1	Problem Formulation	16
5.2	Implementation Details	16
5.2.1	State Representation:	17
5.2.2	Neighborhood Structure	17
5.2.3	Objective Function: Modularity	17
5.3	Results	17
6	A new custom cooling schedule	18
7	Conclusion	19

1 Introduction

Combinatorial optimization is the process of finding an optimal solution from a finite set of discrete possibilities, ie where the set of feasible solutions is discrete or can be reduced to a discrete set.

Many real world problems especially in Operations Research, like the Traveling salesman problem, graph partitioning are combinatorial optimization problems that are:

- often very difficult to solve due to the very large number of possible combinations (often **NP hard**), hence computationally very expensive.
- Also these problems often have **multiple local optimas** where standard techniques like gradient descent might get stuck in.

To solve such problems we will use the Simulated Annealing algorithm which is based on the **annealing** process used in **metallurgy**, where a metal is heated to a high temperature quickly and then gradually cooled. At high temperatures, the atoms move fast. Moreover, when the temperature is reduced, their kinetic energy decreases as well. At the end of the annealing process, the atoms fall into a more ordered state.

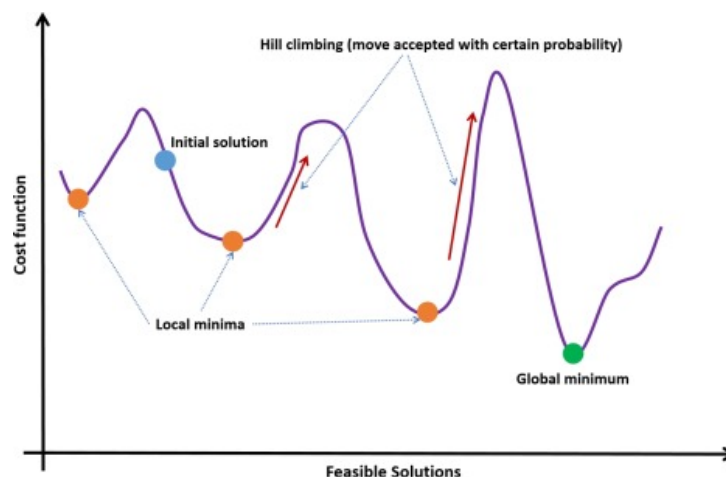
Similarly, in Simulated Annealing, a search process starts with a high energy state (an initial solution) and gradually lowers the temperature (a control parameter) until it reaches a state of minimum energy (the optimal solution).

In this project, we have implemented an object oriented SA framework in Python and have applied it to three distinct domains:

1. **Logistics:** Traveling Salesperson Problem (TSP)
2. **Finance:** Portfolio Optimization
3. **Network Science:** Community Detection (Modularity Maximization)

2 Methodology

2.1 What is Simulated Annealing?



1. **Setup:** Start with a random initial solution (S) of the problem and set a high starting temperature (T). This temperature determines the algorithm's willingness to accept solutions that have a higher cost.
2. **Create a Neighbor:** Generate a new candidate solution (S') by making a tiny adjustment to the current solution (choose a neighbour of the current solution)
3. **Compare Quality:** Calculate the objective of the new solution.
 - If the new solution (S') is better (lower cost) than the current one, accept it immediately.
4. **Accepting a worse solution:** If the new solution is *worse*, do not reject it automatically. Instead, introduce a probability of accepting that worse solution. The chance of accepting a worse solution depends on how much worse it is (ΔE) and how high the current temperature (T) is. This is inspired by the Boltzmann distribution in physical annealing which is given by : -

$$P(\text{accept}) = e^{-\frac{\Delta E}{T}}$$

High temperatures allow for more bad moves (exploration), while low temperatures will restrict them.

5. **Cool Down:** After each round, lower the temperature (T) based on a specific schedule (for example, multiplying it by 0.99). This makes the algorithm stricter reducing the probability of accepting worse moves and allowing the system to settle into a minimum.
6. **Stop:** Repeat the cycle until the temperature drops to near zero, no improvements are made for a while, or a specific number of steps is completed.

In conclusion, the probability of accepting that step is defined by,

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ e^{-\Delta E/T} & \text{if } \Delta E > 0 \end{cases} \quad (1)$$

This criteria is known as the **Metropolis Criterion**

2.2 Advantages of Simulated Annealing

- **Global Optimization:** Unlike greedy algorithms (e.g., Gradient Descent) which get trapped in the nearest local minimum, SA accepts worse solutions with a certain probability, allowing it to explore the search space and escape local optima.
- **Versatile:** It does not require the objective function to be differentiable or continuous. This makes it applicable to a wide range of problems, including discrete (TSP), continuous (Portfolio Optimization), and structural (Graph) domains.
- **Efficiency on NP-Hard Problems:** For complex problems where finding the exact solution requires exponential time (exhaustive search), SA offers a significant speed advantage. It provides a high-quality approximate solution in a reasonable timeframe, making it practical for large-scale optimization tasks.

2.3 Implementation Architecture

To compute the solutions of these problems, we have implemented reusable classes in Python for the problem definitions. This allows us to utilize our solver capable of handling completely different problem instances.

2.3.1 The Abstract State Class

We defined an abstract base class `State` that will be used as a standard interface for any problem we wish to solve. The Subclasses must implement:

- `cost()`: Computes the objective function value for a given state
- `get_neighbour()`: Generates a single random candidate move (small tweak)
- `update(move)`: Applies the proposed move to the current solution if accepted on basis of probability function
- `cost_change(move)`: Efficiently calculates the difference in cost (ΔE) without re-computing the full objective.

Additionally, to support deterministic local search (Steepest Descent), we included methods to manage the full neighborhood:

- `get_all_neighbours()`: Generates a priority queue (using heapdict) containing all possible moves from the current configuration.
- `get_affected_moves()`: Updates the neighborhood queue by determining which moves need re-evaluation after a state change.

```

1 class State():
2     def __init__(self):
3         self.state = []
4         pass
5
6     def cost(self, state) -> float:
7         pass
8
9     def get_neighbour(self) -> tuple:
10        pass
11
12    def update(self, move : tuple) -> None:
13        pass
14
15    def cost_change(self, move : tuple) -> float:
16        pass
17
18    def get_all_neighbours(self) -> heapdict:
19        pass
20
21    def get_affected_moves(self, move : tuple, queue : heapdict) ->
heapdict:
22        initial_state = self.state.copy()
23        self.update(move)
24        del queue
25        queue = self.get_all_neighbours()
26        self.state = initial_state
27        return queue

```

2.3.2 The Annealer Class

The `Annealer` class contains the core logic. It maintains the temperature, handles the cooling schedule (Linear, Exponential, or Logarithmic), and executes the loop based on our algorithm.

1. Initialization

The constructor initializes the solver with a specific problem instance (`state`), the starting temperature, and the chosen cooling schedule strategy.

```
1 def __init__(self, state : State, initial_temp : float,
2   temperature_schedule : str | Callable[[float, int, float], float],
3   scheduling_constant : float):
4     self.state = state
5     self.temperature = initial_temp
6     self.step = 0
7     self.temperature_schedule = temperature_schedule
8     self.scheduling_constant = scheduling_constant
9     self.optimal_state = self.state.state
```

2. The Metropolis Criterion

This calculates the acceptance probability using the Boltzmann distribution. If the temperature is zero, it strictly rejects energy increases (pure greedy behavior).

```
1 def calc_prob(self, energy_change : float) -> float:
2     if self.temperature == 0:
3         prob = 0
4     else:
5         prob = math.exp(- energy_change / self.temperature)
6     return prob
```

3. Cooling Schedule Management

The `schedule_step` method will apply the selected cooling curve (Linear, Exponential, or Logarithmic) to update the current temperature after every iteration.

```
1 def schedule_step(self) -> None:
2     if self.temperature_schedule == 'linear':
3         self.linear_schedule()
4     elif self.temperature_schedule == 'exponential':
5         self.exponential_schedule()
6     elif self.temperature_schedule == 'logarithmic':
7         self.logarithmic_schedule()
8     else:
9         self.temperature = self.temperature_schedule(self.
10            temperature, self.step, self.scheduling_constant)
```

4. The Annealing Step

This method will generate a neighbor, calculate the cost difference (ΔE), and decides whether to update to the new state based on the probability returned by `calc_prob`.

```
1 def anneal_step(self) -> float:
2     neighbour_move = self.state.get_neighbour()
3     del_E = self.state.cost_change(neighbour_move)
4     if (del_E <= 0):
5         self.state.update(neighbour_move)
6     else:
7         prob = self.calc_prob(del_E)
8         if (random.random() <= prob):
9             self.state.update(neighbour_move)
10        else:
11            del_E = 0
12        self.step += 1
13        self.schedule_step()
14    return del_E
```

5. The Main Optimization Loop

The `anneal` method runs the full simulation. Apart from the logic of the algorithm used in SA and discussed earlier in this report, we are going to introduce two additional mechanisms to improve the overall results within the main optimization loop:

1. **Iterative Reheating:** Instead of a single run, the algorithm performs multiple consecutive runs. In each subsequent run, the initial temperature is dampened by a reset factor (e.g., $T_{init}^{(k+1)} = 0.5 \times T_{init}^{(k)}$). This stops the solver from settling into a single solution area too early. Throughout these repeated runs, the system tracks and saves the absolute best result found.
2. **Post Annealing Greedy descent (Quench Phase)** Upon the completion of the probabilistic annealing phase, we will run a deterministic **Greedy Descent**. In this phase, only the improving moves ($\Delta E < 0$), ensuring that the solution settles into the exact minimum of the current neighborhood that was recognised by the annealing phrase.

```

1  def anneal(self, steps = None, stop_temp = None,
2  unchanged_threshold = 100, initial_temp = None, reset_temp = 0.5,
3  n_runs = 10, local_search = False) -> State:
4      initial_step = self.step
5      cost = self.state.cost(None)
6      best_cost = cost
7      best_state = self.state.state.copy()
8
9      if initial_temp is not None:
10         self.temperature = initial_temp
11
12         initial_temp = self.temperature / reset_temp
13
14         for i in range(n_runs):
15             initial_temp = reset_temp * initial_temp
16             self.temperature = initial_temp
17             self.step = 0
18
19             unchanged_steps = 0
20
21             while True:
22                 del_E = self.anneal_step()
23
24                 cost += del_E
25
26                 if cost < best_cost:
27                     best_cost = cost
28                     best_state = self.state.state.copy()
29                     unchanged_steps = 0
30                 else:
31                     unchanged_steps += 1
32
33                 if (steps is not None) and ((self.step - initial_step)
34                 >= steps):
35                     break
36                 elif (stop_temp is not None) and (self.temperature <=
37                 stop_temp):
38                     break
39                 elif (unchanged_steps >= unchanged_threshold):
40                     break
41
42             print(f"Run {i+1} / {n_runs}")

```

```
40         print(f"Temperature : {initial_temp:0.4f} Best cost : {  
best_cost}\n")  
41  
42         self.optimal_state = best_state.copy()  
43         self.state.state = best_state.copy()  
44         cost = best_cost  
45  
46         if local_search:  
47             best_cost = self.greedy_search()  
48  
49             print(f"Final local search")  
50             print(f"Best cost : {best_cost}\n")  
51  
52             self.optimal_state = self.state.state  
53  
54         return self.state
```

6. Deterministic Greedy Descent

As introduced earlier, this will be a helper method that will run the descent to the minimum. It utilizes the priority queue (which we formed using the `get_all_neighbours`) to continuously select the absolute best improving move until a local minimum is reached.

```
1  def greedy_search(self):  
2      final_cost = self.state.cost(None)  
3      queue = self.state.get_all_neighbours()  
4      move, del_E = queue.popitem()  
5  
6      while (del_E < 0 and len(queue) > 0):  
7          queue = self.state.get_affected_moves(move, queue)  
8          self.state.update(move)  
9          final_cost += del_E  
10  
11         move, del_E = queue.popitem()  
12  
13     return final_cost
```

Now we are ready to implement this into various different problems.

3 Application I: Traveling Salesperson Problem (TSP)

3.1 Problem Context

The Traveling Salesman Problem (TSP) is a combinatorial optimization problem that aims to find the shortest cyclic path through a graph that visits each node exactly once. The classic example is that of a traveling salesman who wants to visit all cities exactly once and return back to the starting point and we have to find the shortest path possible.

It is a very important problem since it has many real world applications such as vehicle routing, robotic arm movements, network routing, path planning, etc. Any problem which involves movement to all states once and returning back to original state can be modelled as a TSP.

3.2 Limitations of Classical Optimization Methods

Finding an exact solution to TSP is an NP Hard problem and the time complexity is exponential in the number of nodes. Heuristic algorithms such as Nearest Neighbour algorithm and Greedy algorithm may not find a very good solution.

Hence, we can use Simulated Annealing to find a good approximate solution. This provides a good balance between speed and optimality. It is faster than exact methods and also provides better solutions than basic heuristics like nearest neighbour and greedy algorithm.

3.3 Implementation Details

3.3.1 Path Representation

To solve TSP using simulated annealing, we made a subclass of the base state class and implemented the specific details of TSP in the subclass. The graph is stored as a variable in the class. The current path is stored in the self.state variable.

We create an initial path using the nearest neighbour algorithm.

```

1  class TSP(Simulated_annealing.State):
2  def __init__(self, graph : np.ndarray):
3      self.graph = graph
4      self.n_nodes = graph.shape[0]
5
6      unexplored = list(range(self.n_nodes))
7      curr = 0
8
9      self.state = []
10     self.state.append(curr)
11     unexplored.remove(curr)
12
13     for i in range(1, self.n_nodes):
14         curr = unexplored[np.argmin(self.graph[curr, unexplored])]
15         self.state.append(curr)
16         unexplored.remove(curr)

```

3.3.2 Neighborhood Structure

The neighbourhood of a path is defined by the set of all paths that we get after performing a 2-opt move. A 2-opt move is a move that switches 2 edges of the path. We cut the path at 2 edges and get 2 segments. Then, we reverse one of the segments and join it

back to the other segment. This effectively cuts 2 of the edges and creates 2 new edges by joining the nodes on opposite sides.

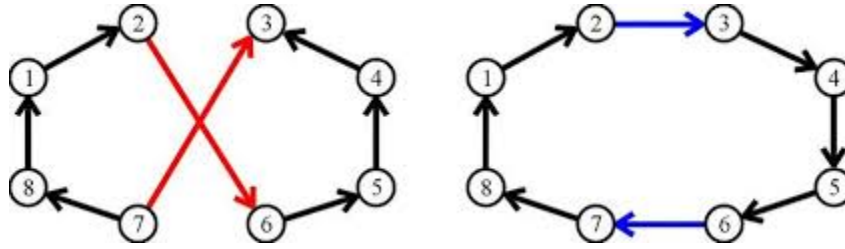


Figure 1: Example of a 2-opt move

This is a good metric for the local neighbourhood because the new path is very similar to the original one and it can eliminate long inefficient zigzags or crossings. Hence, we define any move to be a tuple of numbers representing the indices of edges to cut for a 2-opt move. The `get_neighbour`, `cost_change` and `update` functions are defined accordingly

```

1  def get_neighbour(self):
2      i = random.randrange(0, self.n_nodes)
3      j = i
4
5      while (j == i):
6          j = random.randrange(0, self.n_nodes)
7
8      i, j = min(i, j), max(i, j)
9
10     while (i == 0 and j == self.n_nodes - 1):
11         i = random.randrange(0, self.n_nodes)
12         j = i
13         while (j == i):
14             j = random.randrange(0, self.n_nodes)
15
16     i, j = min(i, j), max(i, j)
17
18     return (i, j)
19
20     def update(self, move):
21         i, j = move
22         i, j = min(i, j), max(i, j)
23
24         self.state = self.state[0 : i] + list(reversed(self.state[i : j
+1])) + self.state[j+1:]

```

3.3.3 Objective Function

The cost (energy) function is the total distance of the path. This is the standard and obvious cost that we must minimise.

```

1  def cost(self, state):
2      if state is None:
3          state = self.state
4
5      cost = 0
6      for i in range(0, self.n_nodes):
7          cost += self.graph[state[i], state[(i+1) % self.n_nodes]]
8
9      return cost

```

3.3.4 Greedy Search

For the greedy search, we use a new class called `Move_Costs` instead of implementing a queue in order to save time by vectorizing operations. It stores the cost of all possible moves in a numpy array and updates the required elements as required by using fast vectorized operations. The `Move_Costs` class is a subclass of `np.ndarray` and is same as the array class but it contains an extra method for `popitem()` which returns the minimum cost move. This greatly reduces the time taken for running the greedy local search algorithm

```

1  class Move_Costs(np.ndarray):
2      def __new__(cls, input_array):
3          # Cast the existing array to be an instance of this class
4          obj = np.asarray(input_array).view(cls)
5          return obj
6
7      def popitem(self):
8          row, col = np.unravel_index(np.argmin(self), shape = self.
shape)
9          return (row, col), self[row][col]
10
11
12  def get_all_neighbours(self):
13      n = self.n_nodes
14      state_arr = np.array(self.state)
15
16      move_matrix = np.full((n, n), np.inf)
17
18      # Get indices for Upper Triangle
19      U, V = np.triu_indices(n, k=1)
20
21      # Vectorized Cost Calculation
22      prev_u_indices = (U - 1) % n
23      curr_u_indices = U
24      curr_v_indices = V
25      next_v_indices = (V + 1) % n
26
27      node_prev_u = state_arr[prev_u_indices]
28      node_curr_u = state_arr[curr_u_indices]
29      node_curr_v = state_arr[curr_v_indices]
30      node_next_v = state_arr[next_v_indices]
31
32      old_costs = self.graph[node_prev_u, node_curr_u] + self.graph[
node_curr_v, node_next_v]
33      new_costs = self.graph[node_prev_u, node_curr_v] + self.graph[
node_curr_u, node_next_v]
34
35      deltas = (new_costs - old_costs).astype(float)
36
37      # Edge case: wrap-around move (0, N-1) is usually invalid/0
38      mask_edge_case = ((U == 0) & (V == n - 1))
39      deltas[mask_edge_case] = np.inf
40
41      # Fill Matrix
42      move_matrix[U, V] = deltas
43      move_matrix = Move_Costs(move_matrix)
44
45      return move_matrix

```

3.4 Results and Performance

On a randomly generated graph with 1000 nodes with distances sampled from a uniform distribution with values 1 to 100, the algorithm ran in a few seconds and gave a good solution. There was a significant improvement over the initial nearest neighbour solution.

```
[108] ✓ 1.5s
... Temperature : 1.5625 Best cost : 1292

Run 8 ↘ 15
Temperature : 0.7812 Best cost : 1292

Run 9 ↘ 15
Temperature : 0.3906 Best cost : 1276

Run 10 ↘ 15
Temperature : 0.1953 Best cost : 1266

Run 11 ↘ 15
Temperature : 0.0977 Best cost : 1266

Run 12 ↘ 15
Temperature : 0.0488 Best cost : 1266

Run 13 ↘ 15
Temperature : 0.0244 Best cost : 1266

Run 14 ↘ 15
Temperature : 0.0122 Best cost : 1259

Run 15 ↘ 15
Temperature : 0.0061 Best cost : 1259

Final local search
Best cost : 1138.0

Completed Annealing
```

Figure 2: Results of Simulated Annealing to solve TSP on a random graph

The local greedy search further improved the solution and ensured that it was a local optimum. This step turned out to be crucial as simulated annealing would often find a good valley but fail to find the absolute local minimum

3.5 Limitations and Tradeoffs

Simulated annealing doesn't always find the global optimum. In order to do that, a large number of steps are needed. There are other heuristic algorithms such as LKH algorithm which give very good solutions in short time.

3.6 Conclusion

In conclusion, Simulated Annealing can be used for solving TSP as it gives a good solution in a reasonable amount of time. However other heuristic algorithms may also be used.

4 Application II: Portfolio Optimization

4.1 Problem Context

Portfolio Optimization is the process of finding the best combination of assets to maximize the returns and minimize the risk usually calculated using Sharpe ratio for stocks. Simple portfolio Optimization where we need to minimize risk(variance) can be done by solving Linear Equations where there is no transaction/other costs and no constraints on weights (except they must sum to one).

But when we add a simple constraint like weight of any asset is limited to say 30% the solution space becomes non-convex and cannot be solved using Linear Programming but still Quadratic Programming can solve this problem using advanced methods. We will try to solve this problem using Extended Simulated Annealing.

Note: Anywhere written Classical Optimization Method refers to Markowitz mean-variance portfolio optimization.

4.2 Limitations of Classical Optimization Methods

The Markowitz mean-variance portfolio:

$$\min_w w^T C w \quad \text{s.t.} \quad \sum_i w_i = 1, \mu^T w = r$$

where w represents the vector of weights of all assets in the portfolio, C represents the Covariance matrix between returns of all assets. μ is the expected returns vector and r is the total return vector. This system can be solved using Linear Programming by converting the objective function to a linear one by minimizing the maximum deviation. But if we add a new inequality constraint $w_i \leq 0.3 \quad \forall i$, then it can no longer be converted to a simple Linear Program and we need Quadratic Programming to solve it. Also we will maximize the sharpe ratio rather than minimizing the risk. Introducing sharpe ratio also removes the possibility of using a Quadratic Programming solution.

$$\text{Sharpe Ratio} = \frac{\mu^T w - r_f}{\sqrt{w^T C w}}$$

where r_f is the risk free rate

4.3 Why Simulated Annealing is used

As stated earlier Quadratic Programming is fully capable and efficient for solving these type of problems but simulated annealing is used because of its flexibility. If we implement Quadratic Programming once for this constrained Markowitz Portfolio and if we need to implement another constraint say, cardinality constraint (i.e., only k assets to be chosen out of n total), then Quadratic Programming fails whereas we need to just add one more condition in simulated annealing and that's it. Also quadratic programming might fail when there are non-convex or discrete spaces but simulated annealing gives us the freedom to search those spaces too.

4.4 Portfolio Representation

The portfolio is represented using a `numpy` array which contains all the weights of each respective asset. `PortfolioState.state` contains the array of weights at any point in the process.

4.5 Constraint Handling

There are two major constraints that need to be taken care of for this - weight should be positive (no - short selling) and no weight is more than maximum allowed weight.

```

1  def apply_move(self, w, move):
2      # delta weight to be moved from ith asset to jth asset.
3      i, j, delta = move
4      w2 = w.copy()
5
6      delta = min(delta, w2[i])
7      delta = min(delta, self.max_w - w2[j])
8
9      if delta < 0:
10         return w.copy()
11
12     w2[i] -= delta
13     w2[j] += delta
14     return w2

```

Code Block 1: Constraint Handling

The condition `delta = min(delta, w2[i])` ensures that `w2[i]` doesn't go negative because `delta` will be always less than `w2[i]` thanks to this condition. And the condition `delta = min(delta, self.max_w - w2[j])` ensures that `w2[j]` doesn't go more than `max_w` which represents the maximum weight allowed for any asset..

4.6 Neighborhood Structure

For changing the state, we find the neighbourhood states by choosing two assets at random `i, j` and shifting a small weight `delta` from asset `i` to asset `j`.

```

1  def get_neighbour(self):
2      i, j = np.random.choice(self.n, 2, replace=False)
3      w = self.state
4
5      max_delta = min(w[i], self.max_w - w[j])
6      if max_delta <= 0:
7         return self.get_neighbour() # retry
8      delta = np.random.uniform(0, max_delta * 0.2)
9
10     return (i, j, delta)

```

Code Block 2: Neighbourhood Structure

4.7 Objective Function

The objective function is the sharpe ratio of a specific portfolio of weights. Sharpe ratio represents how much returns you can get above the risk free rate per unit of risk you have to take.

$$\text{Sharpe Ratio} = \frac{\mu^T w - r_f}{\sqrt{w^T C w}}$$

where r_f is the risk free rate

```

1  def sharpe(self, w):
2      r = self.portfolio_return(w)
3      s = self.portfolio_risk(w)
4      return 0 if s == 0 else (r - self.rf) / s
5
6  def cost(self, state):
7      # cost = maximize sharpe
8      w = self.state if state is None else state
9      return -self.sharpe(w)

```

Code Block 3: Objective and Cost function

4.8 Results

The following graph shows the cost (-sharpe) for a specific run. The first half part of the steps is the exploration phase where the temperature is high enough to allow a lot of up's and downs. But after some refinement the graph starts to go down steeply - because it sees the minimum cost till then and it continues from that point the greedy search. After a specific point it stabilizes the greedy search also reaching a global minimum.

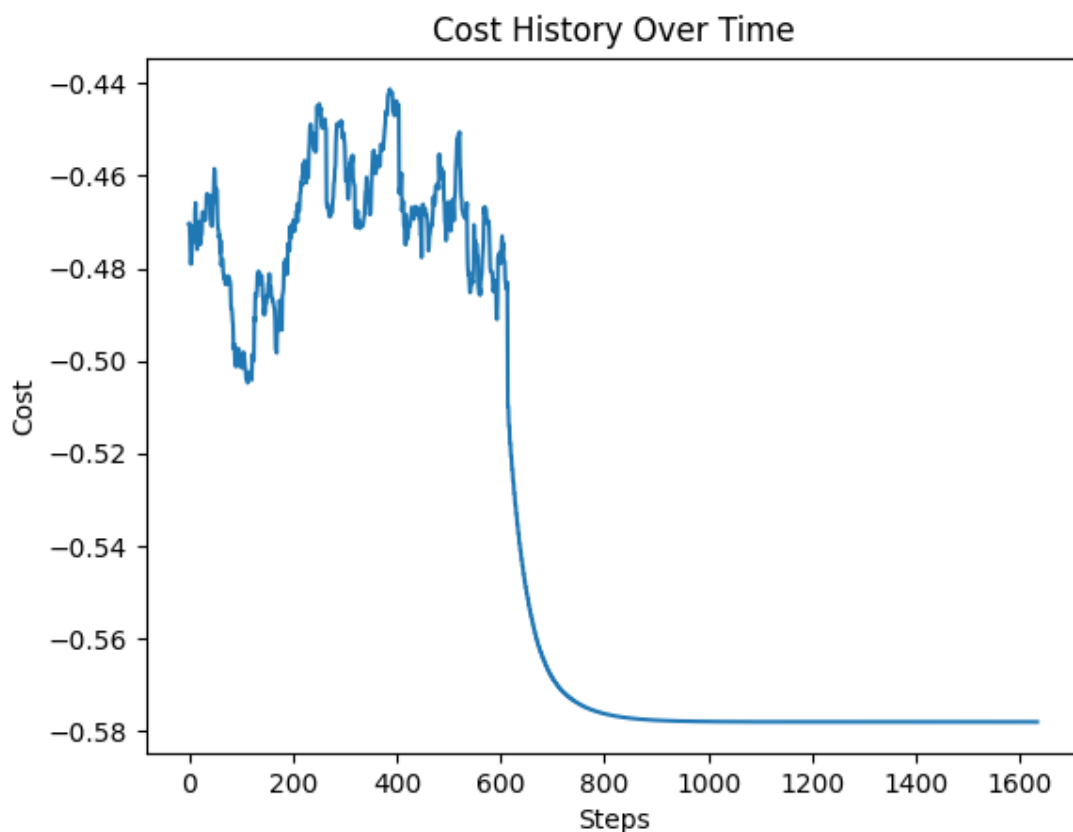


Figure 3: Steps v/s Cost History

Parameters used for this:

- Initial Temperature = 1 (Because of the magnitude of energy change)
- Scheduling Constant = 0.001

- Cooling Schedule = Exponential

Results: (Sharpe Ratio)

- Simulated Annealing Result = 0.504
- Greedy Search Result = 0.577

4.9 Impact of the Quench Phase

A unique addition to our solver was the **Quench** phase. In test cases, the Quench phase improved the final solution cost by an average of 5-10% compared to stopping immediately after the temperature reached zero. This confirms that while SA is good at global exploration, a final deterministic gradient descent is necessary for final global optimum.

5 Application III: Community Detection

5.1 Problem Formulation

A community is formally defined as a subset of nodes within a network where the **internal connections are stronger or more frequent** than the external connections to other nodes in the network. The strength of a community partition is quantified using **Modularity** (Q), which represents the strength of division into communities. It allows us to assess a 'good' community distribution.

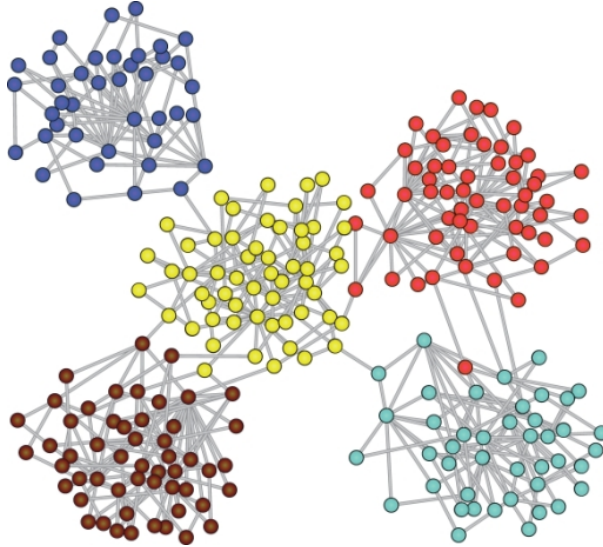


Figure 4: Visual representation of a network with distinct community structures.

The formal equation for Modularity for a partition $\mathcal{C}$ is:

$$Q(\mathcal{C}) := \frac{1}{2m} \sum_{c \in \mathcal{C}} \sum_{u \in c} \sum_{v \in c} \left(A_{uv} - \frac{k_u k_v}{2m} \right) \quad (2)$$

Where:

- $2m$ is the total number of edges in the network (sum of degrees).
- A_{uv} is the weight of the edge between nodes u and v
- k_u and k_v are the degrees of nodes u and v , respectively.
- The term $\frac{k_u k_v}{2m}$ represents the **expected number of edges** between u and v in the random baseline null model.

5.2 Implementation Details

Our target is to find community distribution such that the modularity is maximised.

5.2.1 State Representation:

```

1  def __init__(self, adj_list):
2      self.adj = adj_list
3      self.nodes = list(self.adj.keys())
4      self.m_2 = sum(sum(n.values()) for n in self.adj.values())
5      self.adj_lists = {n: list(self.adj[n].keys()) for n in self.
nodes}
6      self._state = {node: random.randint(0, len(self.nodes)) for
node in self.nodes}
7      self.node_vol = {node: sum(neighbors.values()) for node,
neighbors in self.adj.items()}

```

Code Block 4: State Initialization

5.2.2 Neighborhood Structure

To generate a neighbor state, we select a random node u and propose moving it to the community of one of its connected neighbors v .

```

1  def get_neighbour(self) -> tuple:
2      node_u = random.choice(self.nodes)
3      neighbors = self.adj_lists[node_u]
4
5      # Pick a random neighbor v and try moving u to v's community
6      neighbor_v = random.choice(neighbors)
7      new_comm = self._state[neighbor_v]
8
9      return (node_u, new_comm)

```

Code Block 5: Generating Neighbor Moves

5.2.3 Objective Function: Modularity

We maximize **Modularity** (Q), which measures the strength of division of a network into different communities.

```

1  def cost(self, state=None) -> float:
2      modularity = 0
3      active_communities = set(self._state.values())
4      for c in active_communities:
5          if self.m_2 > 0:
6              modularity += (self.comm_edges[c] / self.m_2) - (
self.comm_vol[c] / self.m_2) ** 2
7      return -modularity

```

Code Block 6: Objective Function

5.3 Results

The algorithm was tested on a standard benchmark graph from SNAP dataset of facebook users having 4039 nodes and over 17,000 edges.

```

Loading Graph from facebook.csv...
Graph loaded with 4039 nodes.
Running Annealer...
Run 1 / 5
Temperature : 1.0000  Best cost : -0.10586905801248409

Run 2 / 5
Temperature : 0.7000  Best cost : -0.400378082316067

Run 3 / 5
Temperature : 0.4900  Best cost : -0.6644626138810753

Run 4 / 5
Temperature : 0.3430  Best cost : -0.7658085488907014

Run 5 / 5
Temperature : 0.2401  Best cost : -0.7968854099328285

--- Final Results ---
Final Modularity: 0.7969
Total Execution Time: 4.2171 seconds
Total Number of Communities: 118

```

Parameters used for this:

- Initial Temperature = 1.0
- Scheduling Constant = 0.2
- Cooling Schedule = Exponential

Results: (Modularity Q)

- Simulated Annealing Result = 0.79
- Greedy Search (Louvain) Result = 0.85

The Simulated Annealing approach achieved a result comparable to the standard Louvain (a popular method of community detection) method, demonstrating the effectiveness in lesser or equal time. (time calculation is demonstrated in code). Also the communities detected by both algorithm show Normalised Mutual Info = 0.93 ie the communities detected are 93% similar.

6 A new custom cooling schedule

We implemented a new custom cooling schedule called `wavy_schedule` which uses an exponential decay function combined with a sinusoidal *reheating* schedule. The function is given by:

$$T(t) = k \cdot b^t \cdot \cos^2(a \cdot \pi \cdot t)$$

where t is the step number and a , b and k are constants

This ensures that the temperatures decreases exponentially while also allowing a sinusoidal *reheating*. The advantage of this is that there are periods of high temperature which allows the algorithm to explore solutions followed by periods of low temperature which prioritizes minimization of cost. This allows the algorithm to first explore, find local minima, explore again and find other minima and so on and finally settle on the solution with lowest cost.

```
1 def wavy_schedule(temp, step, const):  
2     a = 100  
3     b = 1 - 0.0001  
4     y = const * (math.cos(a * math.pi * step)**2 * math.exp(math.log(b)  
5         * step))  
6     return y
```

However, we found that this schedule did not improve the cost by a significant amount especially after implementing the multiple runs feature and hence, we did not use it.

7 Conclusion

In this project, we successfully developed a general-purpose Simulated Annealing solver in Python. By leveraging object-oriented principles, we applied the exact same optimization engine to discrete permutation problems (TSP), continuous constrained problems (Portfolio), and graph-theoretical problems (Community Detection). The results demonstrate the versatility of Simulated Annealing as a robust metaheuristic for Operations Research.